

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Carrera: Software (Rediseño)

Asignatura: Aplicaciones Web — 5to Nivel A

FUNDAMENTO TEÓRICO

Acceso a Datos y Patrones de Arquitectura:

Integración ORM, Caché con Redis y Diseño Arquitectónico C4

Período Académico: 2026–2027 PPA

Fecha: 01-05-2026

Índice

1. Gestión de Estado en Aplicaciones Web: <i>Request</i>, <i>Response</i> y <i>Session</i>	2
1.1. La Naturaleza Sin Estado de HTTP y sus Implicaciones.....	2
1.2. Comparativa de Mecanismos de Almacenamiento del Lado del Cliente	2
1.3. Sesiones Escalables con Redis.....	3
1.4. Objetos <i>Request</i> y <i>Response</i> en el Framework del PFC.....	3
2. ORM y Mapeo Objeto-Relacional	4
2.1. El Problema del <i>Impedance Mismatch</i>	4
2.2. Comparativa ORM vs. SQL Puro vs. <i>Query Builder</i>	4
2.3. El Problema N+1 y <i>Lazy/Eager Loading</i>	4
2.4. Migraciones y Gestión del Esquema.....	5
3. Patrones de Arquitectura para Aplicaciones Web Escalables	7
3.1. El Modelo C4 para Documentar Arquitectura de Software	7
3.2. Arquitectura por Capas vs. Microservicios: Análisis para el PFC.....	7
3.3. Event-Driven Architecture (EDA) y Casos de Uso Web.....	8
3.4. Architecture Decision Records (ADR) como Práctica de Gobernanza Arquitectónica	8
4. Caché: Patrones y Medición de Rendimiento	10
4.1. Niveles de Caché en una Aplicación Web	10
4.2. Patrones de Caché: Cache-Aside, Read-Through, Write-Through y Write-Behind	10
4.3. Redis: Estructura de Datos y Comandos Fundamentales	11
4.4. Metodología de Benchmarking de Rendimiento Web.....	11

1. Gestión de Estado en Aplicaciones Web: *Request, Response* y *Session*

1.1. La Naturaleza Sin Estado de HTTP y sus Implicaciones

El Protocolo de Transferencia de Hipertexto (HTTP) fue diseñado como un protocolo sin estado (*stateless*), principio consagrado formalmente en el RFC 7230, que establece que cada par petición-respuesta debe ser independiente y que el servidor no está obligado a retener información sobre transacciones anteriores (**fielding2014**). Esta característica arquitectónica confiere al protocolo ventajas sustanciales en términos de escalabilidad horizontal: dado que no existe contexto almacenado entre peticiones, cualquier servidor disponible dentro de un clúster puede atender cualquier solicitud entrante sin necesidad de enrutar el tráfico hacia una instancia específica.

No obstante, esta misma característica constituye un desafío fundamental para las aplicaciones web modernas que requieren mantener el estado de interacción del usuario a lo largo del tiempo. Escenarios como el carrito de compras en un portal de comercio electrónico, la autenticación de usuarios en un panel administrativo o la persistencia de preferencias individuales demandan mecanismos adicionales que simulen la continuidad de estado sobre un protocolo intrínsecamente amnésico.

Para abordar esta limitación, la industria ha adoptado cuatro mecanismos principales de gestión de estado: las **cookies HTTP**, las **sesiones del servidor**, los **tokens JWT** (*JSON Web Token*) y el almacenamiento en **bases de datos** (**nixon2021**). Las cookies son fragmentos de datos que el servidor envía al cliente mediante la cabecera Set-Cookie y que el navegador reenvía automáticamente en cada solicitud subsiguiente hacia el mismo dominio. Las sesiones del servidor almacenan el estado en el lado del servidor y únicamente envían al cliente un identificador de sesión. Los tokens JWT implementan un enfoque *stateless*: la información de contexto se codifica directamente en el token, firmado criptográficamente, eliminando la necesidad de un almacén de sesiones centralizado.

1.2. Comparativa de Mecanismos de Almacenamiento del Lado del Cliente

El ecosistema de almacenamiento en el navegador ofrece tres opciones ampliamente utilizadas: `localStorage`, `sessionStorage` y las cookies HTTP. La Tabla 1 sintetiza sus características técnicas y perfiles de riesgo de seguridad.

Cuadro 1: Comparativa de mecanismos de almacenamiento en el cliente

Criterio	localStorage	sessionStorage	Cookies HTTP
Capacidad	~5–10 MB	~5–10 MB	~4 KB
Persistencia	Indefinida	Solo pestaña activa	Configurable
Acceso desde JS	Sí	Sí	Solo sin HttpOnly
Envío automático con HTTP	No	No	Sí (mismo dominio)
Riesgo XSS	Alto	Alto	Bajo con HttpOnly
Riesgo CSRF	Nulo	Nulo	Presente (mitigable)

Para el contexto del Proyecto Fin de Curso (PFC) con autenticación de usuarios, se recomienda el uso de tokens JWT almacenados en cookies con los atributos `HttpOnly` y `Secure` activados, ya que este

esquema elimina el acceso al token desde JavaScript —mitigando XSS— y restringe su transmisión a canales cifrados —mitigando ataques de interceptación— (**welling2016**).

1.3. Sesiones Escalables con Redis

En arquitecturas de múltiples instancias de servidor, el almacenamiento de sesiones en el sistema de archivos local genera el problema conocido como *sticky sessions* o *session affinity*: todas las peticiones de un usuario deben enrutarse obligatoriamente hacia el mismo nodo que creó su sesión, lo que limita severamente la eficacia del balanceo de carga y crea puntos únicos de fallo (**nixon2021**).

La solución estándar consiste en centralizar el almacenamiento de sesiones en una capa de datos compartida, siendo Redis la opción más adoptada en la industria debido a su latencia de acceso en el orden de los microsegundos y su soporte nativo de expiración de claves mediante TTL (*Time To Live*). La Tabla 2 compara las tres estrategias de almacenamiento de sesiones.

Cuadro 2: Comparativa de estrategias de almacenamiento de sesiones

Estrategia	Velocidad	Persistencia	Atomicidad	Escalabilidad
Sistema de archivos	Baja	Alta	No nativa	Muy baja
Base de datos relacional	Media	Alta	Sí (ACID)	Media
Redis (en memoria)	Muy alta	Configurable (AOF/RDB)	Sí	Alta

1.4. Objetos *Request* y *Response* en el Framework del PFC

En frameworks como **Laravel** (PHP), los objetos Request y Response encapsulan el ciclo completo de una solicitud HTTP. El objeto `Illuminate\Http\Request` proporciona métodos para acceder a parámetros de consulta (`$request->query('param')`), datos del cuerpo de la petición (`$request->input('campo')`), cabeceras (`$request->header('Authorization')`) y archivos adjuntos (`$request->file('documento')`). La respuesta se construye mediante helpers como `response()->json($datos, 200)`, que generan automáticamente las cabeceras Content-Type apropiadas (**laravel2024**).

El ciclo completo sigue esta secuencia: recepción del HTTP request → middleware de autenticación/autorización → enrutamiento al controlador → ejecución de la lógica de negocio en el servicio → acceso a datos via Repository/ORM → construcción del response JSON o HTML → envío al cliente. En **ASP.NET Core**, este ciclo se implementa mediante el pipeline de middleware configurado en `Program.cs`, y el objeto `HttpContext` centraliza el acceso al Request y al Response dentro de cada handler (**lock2021**).

2. ORM y Mapeo Objeto-Relacional

2.1. El Problema del *Impedance Mismatch*

El término *impedance mismatch* (desajuste de impedancia) describe la fricción conceptual fundamental que existe entre el modelo relacional —basado en tablas, filas, claves foráneas y operaciones de álgebra relacional— y el modelo orientado a objetos —basado en clases, herencia, polimorfismo y asociaciones de objetos— (**fowler2002**). Este desajuste se manifiesta en múltiples dimensiones concretas:

- **Granularidad:** Un único objeto del dominio (p. ej., Cliente) puede requerir el mapeo a múltiples tablas en el modelo relacional (*cliente, dirección, datos_contacto*).
- **Herencia:** El modelo relacional no dispone de un mecanismo nativo para representar jerarquías de herencia entre entidades, lo que obliga a adoptar estrategias de mapeo específicas.
- **Identidad:** En el modelo relacional, dos filas son iguales si comparten la misma clave primaria; en el modelo de objetos, la igualdad puede definirse mediante el método `equals()` o por comparación de referencias en memoria.
- **Asociaciones:** Los *joins* relacionales no tienen un equivalente semántico directo en la navegación de grafos de objetos en memoria.

Para resolver la representación de herencia en ORM, **fowler2002** propone tres estrategias de mapeo canónicas:

1. **Single Table Inheritance (STI):** Todas las subclases se almacenan en una única tabla con una columna discriminadora. Ventaja: consultas simples sin joins. Desventaja: proliferación de columnas nulas. Recomendable cuando las subclases difieren en pocos atributos.
2. **Class Table Inheritance (CTI):** Cada clase de la jerarquía tiene su propia tabla, vinculadas mediante claves foráneas. Ventaja: estructura normalizada. Desventaja: requiere joins para reconstruir objetos concretos.
3. **Concrete Table Inheritance:** Solo las clases concretas (no abstractas) tienen tabla propia, duplicando los atributos heredados. Ventaja: consultas eficientes por tipo. Desventaja: duplicación de datos y complejidad en consultas polimórficas.

2.2. Comparativa ORM vs. SQL Puro vs. Query Builder

La Tabla 3 presenta la comparativa técnica entre las tres estrategias de acceso a datos más utilizadas.

2.3. El Problema N+1 y *Lazy/Eager Loading*

El problema N+1 es uno de los antipatrones de rendimiento más frecuentes en aplicaciones que utilizan ORM. Ocurre cuando se ejecuta una consulta inicial para obtener N registros de una entidad padre, seguida de N consultas individuales adicionales para obtener los datos relacionados de cada registro hijo, resultando en un total de $N + 1$ consultas (**fowler2002**). El costo escala linealmente con el tamaño del conjunto de datos.

El siguiente ejemplo en Laravel ilustra el problema y su solución mediante *eager loading*:

Cuadro 3: Comparativa de estrategias de acceso a datos

Criterio	ORM	SQL Puro	Query Builder
Curva de aprendizaje	Media-Alta	Baja	Baja-Media
Rendimiento (consultas simples)	Medio	Alto	Alto
Rendimiento (consultas complejas)	Bajo (riesgo N+1)	Alto	Medio
Portabilidad entre SGBD	Alta	Baja	Media
Facilidad de migración	Alta	Manual	Parcial
Debugging de queries	Difícil (log)	Directo	Medio

Listing 1: Problema N+1 y solución con *eager loading* en Laravel/Eloquent

```

1 // SIN eager loading      problema N+1 (N+1 queries)
2 $usuarios = User::all();
3 foreach ($usuarios as $usuario) {
4     echo $usuario->pedidos; // Dispara una query por cada usuario
5 }
6
7 // CON eager loading      solucion optima (solo 2 queries en total)
8 $usuarios = User::with('pedidos')->get();
9 foreach ($usuarios as $usuario) {
10     echo $usuario->pedidos; // Datos ya cargados en memoria
11 }

```

El *lazy loading* (carga diferida) recupera las asociaciones únicamente en el momento en que se accede a ellas, siendo adecuado para accesos poco frecuentes. El *eager loading* (carga anticipada) es óptimo cuando se sabe con certeza que los datos relacionados serán requeridos para la mayoría de los registros del conjunto resultante (**lock2021**).

2.4. Migraciones y Gestión del Esquema

Las migraciones de bases de datos son mecanismos de control de versiones aplicados al esquema relacional, que permiten evolucionar la estructura de la base de datos de forma reproducible, auditable y reversible (**fowler2002**). Las principales estrategias son:

- **Migraciones versionadas** (Laravel, Entity Framework Core, Flyway): cada cambio se representa como un archivo numerado secuencialmente con operaciones de transformación (up) y reversión (down). Son la práctica recomendada en producción por su trazabilidad y capacidad de rollback controlado.
- **Generación automática de esquema** (Hibernate hbm2ddl.auto=update): el ORM infiere y aplica cambios directamente desde las entidades del modelo de dominio. Conveniente en desarrollo, pero extremadamente riesgoso en producción por la posibilidad de ejecutar operaciones destructivas (DROP, ALTER) de forma involuntaria.
- **Herramientas dedicadas** (Liquibase): ofrecen control granular, soporte para rollbacks complejos

y gestión de múltiples entornos. Adecuadas para proyectos empresariales con equipos de base de datos independientes.

3. Patrones de Arquitectura para Aplicaciones Web Escalables

3.1. El Modelo C4 para Documentar Arquitectura de Software

El Modelo C4, propuesto por Simon Brown, provee un vocabulario jerárquico y estandarizado para describir la arquitectura de un sistema a través de cuatro niveles de abstracción progresivos, cada uno orientado a una audiencia diferente (**bass2021**):

Nivel C1 — Contexto del Sistema: Muestra el sistema como una caja negra en el centro del diagrama, rodeado de los actores humanos (usuarios finales, administradores) y los sistemas externos con los que interactúa (APIs, servicios de correo, pasarelas de pago). Comprensible por cualquier *stakeholder*, incluyendo interlocutores no técnicos.

Nivel C2 — Contenedores: Descompone el sistema en sus unidades desplegables e independientes: la aplicación web (frontend), el servidor de aplicaciones (backend), la base de datos relacional, el servidor de caché Redis, y servicios opcionales como CDN o colas de mensajes. Para cada contenedor se especifica la tecnología y responsabilidad principal.

Nivel C3 — Componentes: Para el contenedor de mayor complejidad (típicamente el backend), descompone la estructura interna en componentes lógicos: Controladores HTTP, Servicios de negocio, Repositorios de datos, Middleware de autenticación y la capa ORM/Entity. Las dependencias entre componentes se representan explícitamente.

Nivel C4 — Código: Representa las clases y sus relaciones mediante diagramas UML para las partes más críticas. Su uso se recomienda únicamente para componentes de alta complejidad, dada la elevada carga de mantenimiento de este nivel.

La superioridad del Modelo C4 sobre los diagramas UML de alto nivel radica en su enfoque en la comunicación efectiva: proporciona el nivel de detalle apropiado para cada audiencia, utiliza un lenguaje visual consistente y no requiere formación especializada en notación UML para su comprensión (**bass2021**).

3.2. Arquitectura por Capas vs. Microservicios: Análisis para el PFC

La arquitectura monolítica por capas organiza el sistema en tres estratos horizontales con responsabilidades claramente delimitadas: la **capa de presentación** (interfaz de usuario o API REST), la **capa de lógica de negocio** (servicios de dominio, reglas de negocio) y la **capa de acceso a datos** (repositorios, ORM, SQL parametrizado). Esta separación promueve el principio de responsabilidad única a nivel arquitectónico y facilita la comprensión del código en equipos pequeños.

La arquitectura de microservicios descompone el sistema en servicios independientes y desplegables de forma autónoma, cada uno propietario de su propio dominio de datos y comunicándose mediante APIs síncronas (REST, gRPC) o eventos asíncronos (Kafka, RabbitMQ). Si bien ofrece ventajas en escalado independiente de componentes, introduce una complejidad operacional considerable: gestión de la comunicación entre servicios, consistencia eventual de datos distribuidos, trazabilidad distribuida (*distributed tracing*) y las denominadas *distributed systems fallacies* —suposiciones erróneas como que la red es confiable, que la latencia es nula o que el ancho de banda es infinito— (**nygard2018**).

Para un PFC universitario con un equipo de tres personas y un horizonte temporal semestral, la arquitectura monolítica por capas es técnicamente la más adecuada. Este criterio se fundamenta en el

concepto de *team cognitive load*: los microservicios incrementan el coste cognitivo de comprensión del sistema más allá de lo que un equipo pequeño puede gestionar eficientemente (**bass2021**). La escalabilidad futura puede abordarse mediante la refactorización incremental hacia una arquitectura modular (*modular monolith*) antes de considerar la migración completa a microservicios.

3.3. Event-Driven Architecture (EDA) y Casos de Uso Web

La Arquitectura Orientada a Eventos (EDA) es un paradigma de diseño en el que los componentes del sistema se comunican mediante la producción, detección y consumo de eventos, en lugar de mediante llamadas directas síncronas. Sus componentes fundamentales son: los **productores de eventos**, el **event bus** (infraestructura de mensajería como RabbitMQ o Apache Kafka) y los **consumidores de eventos** suscritos a tipos específicos de eventos (**bass2021**).

Los WebSockets representan una instancia particular de EDA aplicada a la comunicación en tiempo real. El protocolo WebSocket se establece mediante un mecanismo de *protocol upgrade* sobre HTTP: el cliente envía una petición con la cabecera Upgrade: websocket; si el servidor acepta, ambas partes establecen un canal TCP bidireccional y persistente (HTTP → WebSocket). A diferencia del modelo petición-respuesta unidireccional de REST, este modelo bidireccional permite que tanto el servidor como el cliente inicien la transmisión de datos en cualquier momento, sin necesidad de *polling* periódico.

Para el contexto del PFC, la EDA se justifica en escenarios donde la baja latencia y la sincronización en tiempo real son requisitos funcionales explícitos: notificaciones instantáneas, actualizaciones de inventario en tiempo real o dashboards con datos en vivo. Para operaciones CRUD convencionales, el modelo REST síncrono sobre HTTP sigue siendo la arquitectura más apropiada por su menor complejidad y más sencilla depuración (**nygard2018**).

3.4. Architecture Decision Records (ADR) como Práctica de Gobernanza Arquitectónica

Los Architecture Decision Records (ADR) son documentos de formato ligero que capturan de manera estructurada las decisiones arquitectónicas significativas adoptadas durante el desarrollo de un sistema, incluyendo el contexto que motivó la decisión, las alternativas consideradas, la opción elegida y sus consecuencias previsibles. El formato estándar fue propuesto por **nygard2018** y ha sido adoptado por equipos de ingeniería de organizaciones de referencia global como práctica sistemática de gobernanza arquitectónica.

El *Guide to the Software Engineering Body of Knowledge* (SWEBOK) en su versión 4.0 reconoce la documentación de decisiones arquitectónicas como parte integral del proceso de diseño de software, señalando que las decisiones no documentadas generan *architectural debt* (deuda arquitectónica) que se manifiesta en mayor costo de mantenimiento y mayor dificultad para incorporar nuevos integrantes al equipo (**washizaki2024**).

Un ADR en formato estándar incluye las siguientes secciones:

1. **Título:** Identificador único (p. ej., ADR-001) y descripción declarativa breve.
2. **Estado:** *Proposed* (propuesta), *Accepted* (aceptada y vigente), *Deprecated* (superada) o *Superseded by ADR-XXX* (reemplazada).
3. **Contexto:** Descripción de la situación actual, el problema a resolver y las restricciones relevantes.

4. **Decisión:** Enunciado claro de la opción elegida con justificación técnica.
5. **Consecuencias:** Efectos positivos esperados, limitaciones asumidas y compromisos (*trade-offs*).

4. Caché: Patrones y Medición de Rendimiento

4.1. Niveles de Caché en una Aplicación Web

Un sistema web de producción implementa múltiples niveles de caché en su pila tecnológica, cada uno optimizado para un tipo específico de dato y patrón de acceso (**fowler2002**; **nygard2018**). La Tabla 4 resume los cuatro niveles principales.

Cuadro 4: Niveles de caché en una aplicación web de producción

Nivel	Dato cacheado	TTL	Invalidación
Caché del navegador	Recursos estáticos (JS, CSS, img)	Largo (días–meses)	Versionado de URL
CDN	Contenido estático/semi-estático	Medio (horas–días)	Purga por evento
Caché de aplicación (Redis)	Resultados de queries, objetos JSON	Corto–medio (seg–min)	Invalidación explícita
Caché de BD (InnoDB pool)	Páginas de datos, planes de consulta	Gestionado por SGBD	Automática (LRU)

4.2. Patrones de Caché: Cache-Aside, Read-Through, Write-Through y Write-Behind

Los cuatro patrones de caché principales difieren en quién es responsable de poblar y actualizar la caché, y en cómo se coordinan las operaciones con el almacén de datos persistente:

Cache-Aside (*Lazy Loading*): La aplicación gestiona la caché de forma explícita: (1) consulta la caché; (2) si hay *cache hit*, retorna el dato; (3) si hay *cache miss*, consulta la BD, almacena el resultado en caché con TTL configurado y lo retorna. Es el patrón más prevalente por su flexibilidad (**nygard2018**).

Read-Through: Un middleware de caché interpuesto gestiona automáticamente el ciclo de lectura. La aplicación siempre consulta la caché como única fuente de datos. Ventaja: simplifica el código de aplicación. Desventaja: mayor acoplamiento con la infraestructura.

Write-Through: Cada escritura en la BD se replica sincrónicamente en la caché, garantizando coherencia inmediata. Ventaja: datos siempre actualizados. Desventaja: latencia adicional en escrituras.

Write-Behind (*Write-Back*): Las escrituras se realizan primero en la caché y se persisten de forma asíncrona. Maximiza el rendimiento de escritura, pero introduce riesgo de pérdida de datos ante fallos de la caché antes de la persistencia.

Cache Stampede: Cuando múltiples peticiones concurrentes experimentan un *cache miss* simultáneo, todas consultan la BD al mismo tiempo, generando sobrecarga súbita. Las soluciones más comunes son el uso de **mutex** (bloqueo de la primera petición generadora) o la técnica de **probabilistic early expiration**, que renueva proactivamente la caché antes de su expiración con una probabilidad inversamente proporcional al tiempo restante de vida de la clave (**nygard2018**).

4.3. Redis: Estructura de Datos y Comandos Fundamentales

Redis (*Remote Dictionary Server*) es un almacén de datos en memoria de código abierto que soporta múltiples estructuras de datos nativas más allá del simple par clave-valor, diferenciándose así de sistemas como Memcached (**nygard2018**). Las estructuras principales y sus casos de uso en el PFC son:

- **Strings:** Almacenamiento de objetos JSON serializados. Escritura con TTL: SET usuario:42 '{"id":42,"nombre": EX 300. Lectura: GET usuario:42.
- **Hashes:** Representación de entidades con campos individuales modificables de forma atómica. HSET pedido:15 estado .enviado"total "89.99". Permite actualizar un campo sin reescribir el objeto completo.
- **Lists:** Implementación de colas FIFO o pilas LIFO mediante LPUSH/RPUSH y LPOP/RPOP. Adecuado para colas de tareas en segundo plano o feeds de actividad.
- **Sets:** Conjuntos de elementos únicos sin orden definido. Útil para *tracking* de usuarios únicos o conjuntos de permisos activos.
- **Sorted Sets:** Conjuntos con puntuación numérica, mantenidos ordenados automáticamente. Ideal para *rankings*, tablas de posiciones y colas de prioridad.

La integración de Redis en el stack del PFC puede realizarse mediante **Predis** (biblioteca PHP compatible con Laravel) o **StackExchange.Redis** para proyectos en ASP.NET Core (**lock2021**).

4.4. Metodología de Benchmarking de Rendimiento Web

La medición del impacto de la caché debe realizarse siguiendo principios de rigor estadístico para garantizar que los resultados sean representativos, reproducibles y libres de sesgos de medición (**nygard2018**). La metodología comprende los siguientes pasos:

1. **Definición de la variable a medir:** Tiempo de respuesta en milisegundos para la consulta principal del PFC, medido con microtime(true) en PHP o System.Diagnostics.Stopwatch en .NET.
2. **Aislamiento de variables:** Ejecutar ambas pruebas (con y sin caché) sobre la misma consulta, el mismo conjunto de datos, el mismo hardware y las mismas condiciones de red.
3. **Número de repeticiones:** Ejecutar un mínimo de $n \geq 10$ repeticiones para cada condición, descartando la primera ejecución (que puede incluir overhead de inicialización), y calcular la media aritmética $\bar{x}$ y la desviación estándar σ .
4. **Cálculo del Speedup:** Aplicar la fórmula:

$$S = \frac{T_{sin}}{T_{con}} \quad (1)$$

donde T_{sin} es el tiempo promedio sin caché y T_{con} es el tiempo promedio con caché activa. Un valor $S > 1$ confirma que la caché mejora el rendimiento; un valor $S > 2$ es el umbral comúnmente aceptado para justificar la complejidad operacional adicional de Redis (**nygard2018**).

5. **Verificación de significancia estadística:** Confirmar que los intervalos de confianza al 95 % ($\bar{x} \pm 2\sigma$) de ambas condiciones no se solapan, indicando que la diferencia observada es estadísticamente significativa y no producto de la variabilidad aleatoria del sistema.

El log de queries del ORM —activado mediante `DB::enableQueryLog()` en Laravel— permite adicionalmente cuantificar la reducción en el número de consultas emitidas a la base de datos, complementando la métrica temporal con una métrica de eficiencia estructural que evidencia el funcionamiento correcto del patrón *cache-aside*.

Referencias

- Bass, L., Clements, P., Kazman, R. (2021). **Software architecture in practice** (4.^a ed.). Addison-Wesley. ISBN: 978-0-13-688080-1.
- Fielding, R., Nottingham, M., Reschke, J. (2014). **Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing**. RFC 7230. Internet Engineering Task Force. <https://tools.ietf.org/html/rfc7230>
- Fowler, M. (2002). **Patterns of enterprise application architecture**. Addison-Wesley. ISBN: 978-0-321-12521-7.
- Laravel. (2024). **Laravel 11.x documentation**. <https://laravel.com/docs/11.x>
- Lock, A. (2021). **ASP.NET Core in action** (2.^a ed.). Manning Publications. ISBN: 978-1-61729-678-7.
- Nixon, R. (2021). **Learning PHP, MySQL JavaScript** (6.^a ed.). O'Reilly Media. ISBN: 978-1-492-06229-9.
- Nygard, M. T. (2018). **Release it!: Design and deploy production-ready software** (2.^a ed.). Pragmatic Bookshelf. ISBN: 978-1-68050-239-8.
- Washizaki, H. (Ed.). (2024). **Guide to the Software Engineering Body of Knowledge (SWEBOK) Version 4.0**. IEEE Computer Society.
- Welling, L., Thomson, L. (2016). **PHP and MySQL Web development** (5.^a ed.). Addison-Wesley. ISBN: 978-0-321-83389-1.